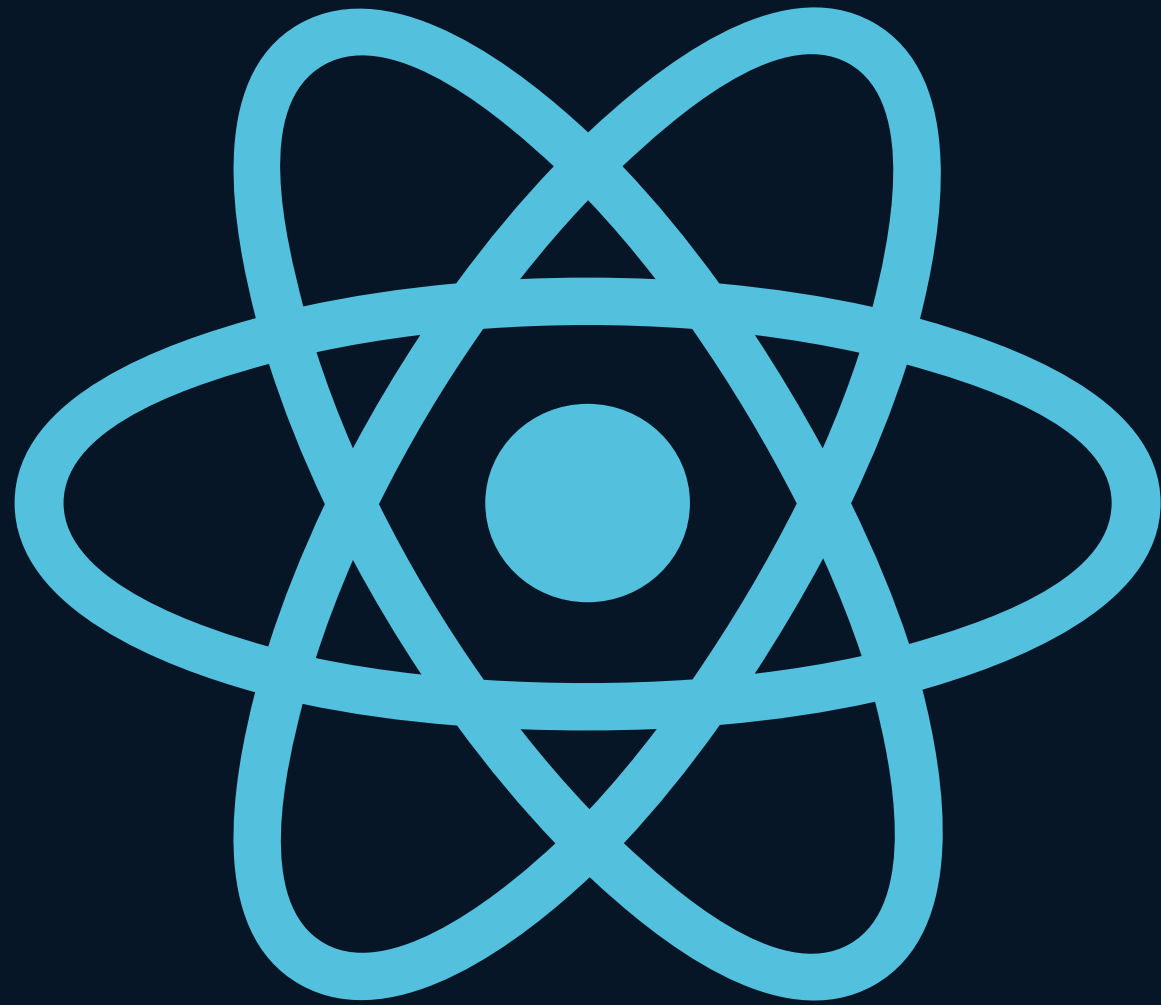


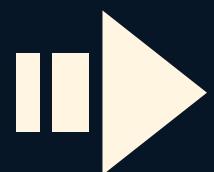
Web dev



4 React Tips



www.abunahid.com





Return functions from handlers

```
export default function App() {
  const [user, setUser] = useState({
    name: "",
    surname: "",
    address: ""
  });

  // First handler
  const handleNameChange = (e) => {
    setUser((prev) => ({
      ...prev,
      name: e.target.value
    }));
  };

  // Second handler!
  const handleSurnameChange = (e) => {
    setUser((prev) => ({
      ...prev,
      surname: e.target.value
    }));
  };

  // Third handler!!!
  const handleAddressChange = (e) => {
    setUser((prev) => ({
      ...prev,
      address: e.target.value
    }));
  };

  // What if we need one more input? Should we create another handler for it?

  return (
    <>
      <input value={user.name} onChange={handleNameChange} />
      <input value={user.surname} onChange={handleSurnameChange} />
      <input value={user.address} onChange={handleAddressChange} />
    </>
  );
}
```



Solution

```
export default function App() {
  const [user, setUser] = useState({
    name: "",
    surname: "",
    address: ""
  });

  const handleInputChange = (field) => {
    return (e) => {
      setUser((prev) => ({
        ...prev,
        [field]: e.target.value
      }));
    };
  };

  return (
    <>
      <input value={user.name} onChange={handleInputChange("name")} />
      <input value={user.surname} onChange={handleInputChange("surname")} />
      <input value={user.address} onChange={handleInputChange("address")} />

      {JSON.stringify(user)}
    </>
  );
}
```



Separate responsibilities

A typical structure for this would be:

- **UI module**, responsible for visual representation only.
- **Logic/Model module**, containing only business logic. An example of this is a custom hook.
- **Lib module**, containing all required utilities for the component.

```
export function ListComponent() {
  // Our local state
  const [list, setList] = useState([]);

  // Handler to load data from the server
  const fetchList = async () => {
    try {
      const resp = await fetch("https://www.url.com/list");
      const data = await resp.json();

      setList(data);
    } catch {
      showAlert({ text: "Something went wrong!" });
    }
  };

  // We want to fetch only on mount
  useEffect(() => {
    fetchList();
  }, []);

  // Handler responsible for deleting items
  const handleDeleteItem = (id) => {
    return () => {
      try {
        fetch(`https://www.url.com/list/${id}`, {
          method: "DELETE"
        });
        setList((prev) => prev.filter((x) => x.id !== id));
      } catch {
        showAlert({ text: "Something went wrong!" });
      }
    };
  };

  // Here we just render our data items
  return (
    <div className="list-component">
      {list.map(({ id, name }) => (
        <div key={id} className="list-component__item">
          {/* We want to trim long name with ellipsis */}
          {name.slice(0, 30) + (name.length > 30 ? "..." : "")}

          <div onClick={handleDeleteItem(id)} className="list-component__icon">
            <DeleteIcon />
          </div>
        </div>
      ))}
    </div>
  );
}
```



We should start by writing our utils that will be used in model and UI modules.

```
export async function getList(onSuccess) {
  try {
    const resp = await fetch("https://www.url.com/list");
    const data = await resp.json();

    onSuccess(data)
  } catch {
    showAlert({ text: "Something went wrong!" });
  }
}

export async function deleteListItem(id, onSuccess) {
  try {
    fetch(`https://www.url.com/list/${id}`, {
      method: "DELETE"
    });
    onSuccess()
  } catch {
    showAlert({ text: "Something went wrong!" });
  }
}

export function trimName(name) {
  return name.slice(0, 30) + (name.length > 30 ? '...' : '')
}
```



Now we need to implement our business logic with a custom hook

```
export function useList() {
  const [list, setList] = useState([]);

  const handleDeleteItem = useCallback((id) => {
    return () => {
      deleteListItem(id, () => {
        setList((prev) => prev.filter((x) => x.id !== id));
      })
    };
  }, []);

  useEffect(() => {
    getList(setList);
  }, []);

  return useMemo(
    () => ({
      list,
      handleDeleteItem
    }),
    [list, handleDeleteItem]
  );
}
```




The final step is to write our UI module and then combine it all together.

```
export function ListComponentItem({ name, onDelete }) {
  return (
    <div className="list-component__item">
      {trimName(name)}

      <div onClick={onDelete} className="list-component__icon">
        <DeleteIcon />
      </div>
    </div>
  );
}

export function ListComponent() {
  const { list, handleDeleteItem } = useList();

  return (
    <div className="list-component">
      {list.map(({ id, name }) => (
        <ListComponentItem
          key={id}
          name={name}
          onDelete={handleDeleteItem(id)}
        />
      ))}
    </div>
  );
}
```



Use objects map instead of conditions

Problem

```
function Account({type}) {  
  let Component = UsualAccount  
  
  if (type === 'vip') {  
    Component = VipAccount  
  }  
  
  if (type === 'moderator') {  
    Component = ModeratorAccount  
  }  
  
  if (type === 'admin') {  
    Component = AdminAccount  
  }  
  
  return (  
    <div className='account'>  
      <Component />  
      <AccountStatistics />  
    </div>  
  )  
}
```




Use objects map instead of conditions

Solution

```
const ACCOUNTS_MAP = {
  'vip': VipAccount,
  'usual': UsualAccount,
  'admin': AdminAccount,
  'moderator': ModeratorAccount,
}

function Account({type}) {
  const Component = ACCOUNTS_MAP[type]

  return (
    <div className='account'>
      <Component />
      <AccountStatistics />
    </div>
  )
}
```



Put independent variables outside of React lifecycle

Problem

```
function useItemsList() {
  const defaultItems = [1, 2, 3, 4, 5]
  const [items, setItems] = useState(defaultItems)

  const toggleArrayItem = (arr, val) => {
    return arr.includes(val) ? arr.filter(el => el !== val) : [...arr, val];
  }

  const handleToggleItem = (num) => {
    return () => {
      setItems(toggleArrayItem(items, num))
    }
  }

  return {
    items,
    handleToggleItem,
  }
}
```



Put independent variables outside of React lifecycle

Solution

```
const DEFAULT_ITEMS = [
  1, 2, 3, 4, 5
]

const toggleArrayItem = (arr, val) => {
  return arr.includes(val) ? arr.filter(el => el !== val) : [...arr, val];
}

function useItemsList() {
  const [items, setItems] = useState(DEFAULT_ITEMS)

  const handleToggleItem = (num) => {
    return () => {
      setItems(toggleArrayItem(items, num))
    }
  }

  return {
    items,
    handleToggleItem,
  }
}
```